

Nexus — Full Session Report

This document covers every change implemented during the session: the authentication system, the real-time discussions feature backed by WebSocket, computed metrics endpoints, database schema, bug fixes, UI cleanup, and presentation notes.

Project	Nexus / Infra Pulse
Stack backend	FastAPI + psycopg2 + bcrypt + WebSocket + TimescaleDB (PostgreSQL)
Stack frontend	React 19 + Vite 8 + Material-UI v7
Branch	feature/login
Backend port	8002 (Python FastAPI)
DB external port	5433 (Docker maps -> 5432 internal)
Total endpoints added	9 REST + 1 WebSocket
Files created	7 (chat_api.py extended, 4 scripts, 2 frontend services, 1 component)
Files modified	8 (App, Layout, Login, Discussions, api.js, theme references, etc.)

Contents

- 1. Architecture overview
- 2. Database schema in full detail
- 3. Authentication system (login, signup, logout)
- 4. Real-time Discussions with WebSocket
- 5. Computed metrics endpoints
- 6. Bug fixes and stabilization
- 7. UI improvements and humanization
- 8. Helper scripts (CLI utilities)
- 9. Setup and run instructions
- 10. Demo / presentation script
- 11. Security notes and known limitations
- 12. File-by-file changelog

1. Architecture overview

Nexus runs as three independent processes that communicate over HTTP and WebSocket:

Component	Port	Stack	Role
Frontend (Vite dev server)	5173	React + MUI	User interface, served as static SPA in dev
Python API	8002	FastAPI + uvicorn	Auth, discussions, metrics, chatbot, WebSocket
Go API (legacy)	8080	Go + GORM	Sensors, events, readings (pre-existing)
PostgreSQL	5433 ext / 5432 int	TimescaleDB	Persistence for all components

Request flow example — user comments on a discussion:

1. User types comment in `Discussions.jsx` → presses Ctrl+Enter
2. `discussionsApi.js` sends `POST /discussions/<id>/comments` to port 8002
3. `FastAPI add_comment()` inserts row in `discussion_comments` table
4. `FastAPI _WSManager.broadcast()` sends JSON to all WebSocket subscribers of that `discussion_id`
5. All connected clients receive the message and append it to the rendered comment list

Why a separate Python API alongside the existing Go one? We kept the Go server untouched because rewriting in Python would be redundant. Python gets the new features (auth, discussions, metrics) which are easier to iterate on; Go keeps serving sensors and event data it was built for. Each frontend module picks the right base URL.

2. Database schema (full)

All tables live in PostgreSQL with the TimescaleDB extension. Migrations are auto-applied on first container start (./migrations/*.sql mounted as /docker-entrypoint-initdb.d).

2.1 Migration 001_init.sql

```
CREATE EXTENSION IF NOT EXISTS timescaledb;

CREATE TABLE locations (
  location_id      SERIAL PRIMARY KEY,
  name             TEXT NOT NULL,
  description      TEXT,
  updated_user_id  INTEGER NOT NULL,
  is_obsolete      BOOLEAN NOT NULL DEFAULT FALSE
);

CREATE TABLE users (
  user_id          SERIAL PRIMARY KEY,
  first_name       TEXT NOT NULL,
  last_name        TEXT,
  email            TEXT NOT NULL UNIQUE,
  role             TEXT NOT NULL,
  password_hash    TEXT NOT NULL,
  created_at       TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_dtm      TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE TABLE sensors (
  sensor_id        SERIAL PRIMARY KEY,
  location_id       INTEGER NOT NULL REFERENCES locations(location_id),
  updated_user_id  INTEGER NOT NULL REFERENCES users(user_id),
  sensor_no        TEXT UNIQUE,
  name             TEXT,
  is_obsolete      BOOLEAN NOT NULL DEFAULT FALSE,
  lower_limit       DECIMAL(10, 2),
  upper_limit       DECIMAL(10, 2),
  updated_dtm      TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE TABLE events (
  event_id         SERIAL PRIMARY KEY,
  location_id       INTEGER NOT NULL REFERENCES locations(location_id),
  sensor_id         INTEGER NOT NULL REFERENCES sensors(sensor_id),
  severity          TEXT NOT NULL,
  status            TEXT NOT NULL DEFAULT 'open',
  metric_value      DECIMAL(10, 2) NOT NULL,
  message           TEXT NOT NULL,
  created_at        TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at        TIMESTAMPTZ,
  resolved_at       TIMESTAMPTZ
);
```

Key columns for the auth and metrics features:

Table.Column	Why it matters
users.email	UNIQUE login identifier
users.password_hash	bcrypt hash (60 chars), never plaintext
users.role	Authorization (admin/user)
events.severity	Used by /metrics/observability for ERROR RATE
events.created_at, events.resolved_at	Difference in /metrics/resolution (avg time to resolve)

events.status	open / acknowledged / resolved
---------------	--------------------------------

2.2 Migration 003_discussions.sql (applied manually)

This migration was added *after* the database was first started, so it wasn't auto-loaded. We applied it manually with `docker exec -i nexus-postgres psql ... < migrations/003_discussions.sql`.

```
CREATE TABLE discussions (
  discussion_id SERIAL PRIMARY KEY,
  event_id INTEGER REFERENCES events(event_id) ON DELETE SET NULL,
  title TEXT NOT NULL,
  status TEXT NOT NULL DEFAULT 'open',
  creator_user_id INTEGER REFERENCES users(user_id) ON DELETE SET NULL,
  author_display TEXT NOT NULL,
  device_label TEXT,
  body TEXT NOT NULL,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE TABLE discussion_comments (
  comment_id SERIAL PRIMARY KEY,
  discussion_id INTEGER NOT NULL REFERENCES discussions(discussion_id) ON DELETE CASCADE,
  user_id INTEGER REFERENCES users(user_id) ON DELETE SET NULL,
  author_display TEXT NOT NULL,
  message TEXT NOT NULL,
  is_system BOOLEAN NOT NULL DEFAULT FALSE,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX discussions_event_id_idx ON discussions(event_id);
CREATE INDEX discussions_status_created_idx
  ON discussions(status, created_at DESC);
CREATE INDEX discussion_comments_discussion_created_idx
  ON discussion_comments(discussion_id, created_at);
```

Design choices for the discussions tables:

- **ON DELETE CASCADE** on comments → deleting a discussion wipes its comments. No orphan rows.
- **ON DELETE SET NULL** for user references → if a user account is deleted, comments stay but lose the user link. Author name is kept in `author_display`.
- **is_system flag** → distinguishes auto-generated entries (e.g. 'Status changed by [admin]') from real user comments.
- **Compound index** on (`status, created_at DESC`) → fast list of open discussions sorted by recency.

2.3 Seeded data

Two scripts ensure the database is usable right after a fresh install:

- `set_password.py` → replaces `placeholder_hash` for the seeded admin with a real bcrypt hash, so login actually works.
- `seed_discussions.py` → inserts three initial discussions with comments. All linked to `creator_user_id = 1` (admin).

3. Authentication system

3.1 End-to-end flow

Login: Login.jsx form → authApi.loginUser() → POST /login → bcrypt verify in chat_api.py → user JSON returned → App.jsx stores it in sessionStorage.nexus_user → Layout renders email + role at top right.

Signup: Same form, mode toggled → POST /signup → bcrypt hashes the new password → INSERT in users table with role='user' → backend returns user data → frontend goes back to login mode with green confirmation message.

Logout: Click avatar → Logout (red) → sessionStorage cleared → App.jsx re-renders the Login screen (useEffect also resets the URL to ?page=Login).

3.2 Backend /login endpoint

```
@app.post("/login", response_model=LoginResponse)
def login(request: LoginRequest):
    with closing(_db_connect()) as conn:
        with conn.cursor() as cur:
            cur.execute(
                "SELECT user_id, email, role, first_name, last_name, "
                "password_hash FROM users WHERE email = %s",
                (request.email.lower().strip(),),
            )
            row = cur.fetchone()

    if not row:
        raise HTTPException(status_code=401, detail="Invalid credentials")

    user_id, email, role, first_name, last_name, password_hash = row

    try:
        ok = bcrypt.checkpw(
            request.password.encode("utf-8"),
            password_hash.encode("utf-8"),
        )
    except (ValueError, TypeError):
        ok = False

    if not ok:
        raise HTTPException(status_code=401, detail="Invalid credentials")

    return LoginResponse(user_id=user_id, email=email, role=role,
                        first_name=first_name, last_name=last_name)
```

Notable defensive choices:

- Same error message ("Invalid credentials") for both wrong-user and wrong-password → prevents account enumeration.
- Try/except around bcrypt.checkpw → legacy/placeholder hashes won't crash the endpoint, they just fail auth.
- Parameterized SQL (%s) → safe against injection.

3.3 Backend /signup endpoint

```
@app.post("/signup", response_model=LoginResponse)
def signup(request: SignupRequest):
    email = request.email.lower().strip()
    pw_hash = bcrypt.hashpw(
        request.password.encode("utf-8"),
        bcrypt.gensalt(),
    ).decode("utf-8")
```

```

with closing(_db_connect()) as conn:
    with conn.cursor() as cur:
        cur.execute("SELECT 1 FROM users WHERE email = %s", (email,))
        if cur.fetchone():
            raise HTTPException(409, "Email already registered")

        cur.execute(
            "INSERT INTO users (first_name, last_name, email, role, "
            "password_hash) VALUES (%s, %s, %s, %s, %s) "
            "RETURNING user_id, email, role, first_name, last_name",
            (request.first_name, request.last_name, email,
             "user", pw_hash),
        )
        user_id, email, role, first_name, last_name = cur.fetchone()
        conn.commit()
    return LoginResponse(...)

```

3.4 Frontend gating in App.jsx

```

function App() {
  const [user, setUser] = useState(() => {
    try {
      const raw = sessionStorage.getItem('nexus_user');
      return raw ? JSON.parse(raw) : null;
    } catch { return null; }
  });
  const isAuthenticated = !!user;

  const handleLogin = (userData) => {
    sessionStorage.setItem('nexus_user', JSON.stringify(userData));
    setUser(userData);
  };

  const handleLogout = () => {
    sessionStorage.removeItem('nexus_user');
    setUser(null);
  };

  useEffect(() => {
    if (!isAuthenticated && params.page !== 'Login') {
      patchParams({ page: 'Login' }, { replaceHistory: true });
    } else if (isAuthenticated && params.page === 'Login') {
      patchParams({ page: 'Dashboard' }, { replaceHistory: true });
    }
  }, [isAuthenticated, params.page, patchParams]);

  if (!isAuthenticated) {
    return (
      <ThemeProvider theme={glassTheme}>
        <Login onLogin={handleLogin} />
      </ThemeProvider>
    );
  }
  // ... rest of app
}

```

Lazy initializer → sessionStorage is read once at mount, not on every render. The **auth gate** → if no user, the rest of the app simply isn't rendered, so unauthenticated users literally cannot navigate to any other page.

4. Real-time Discussions with WebSocket

Goal: when one user posts a comment in a discussion thread, every other user currently viewing the same thread sees it appear instantly — no refresh, no polling needed.

4.1 REST endpoints

Method + path	Purpose
GET /discussions	List all threads with comment count
GET /discussions/{id}	Full thread + comments, ordered ascending
POST /discussions	Create a new thread (status=OPEN, role=user)
POST /discussions/{id}/comments	Add a comment + broadcast through WebSocket
PATCH /discussions/{id}/status	Toggle OPEN/RESOLVED + auto system comment + broadcast
WS /ws/discussions/{id}	Live subscription channel

4.2 WebSocket connection manager

```
class _WSManager:
    def __init__(self):
        self.subs: dict[int, set[WebSocket]] = defaultdict(set)
        self._lock = asyncio.Lock()

    async def connect(self, discussion_id, ws):
        await ws.accept()
        async with self._lock:
            self.subs[discussion_id].add(ws)

    async def disconnect(self, discussion_id, ws):
        async with self._lock:
            self.subs[discussion_id].discard(ws)
            if not self.subs[discussion_id]:
                self.subs.pop(discussion_id, None)

    async def broadcast(self, discussion_id, payload):
        msg = json.dumps(payload, default=str)
        async with self._lock:
            targets = list(self.subs.get(discussion_id, ()))
            dead = []
            for ws in targets:
                try:
                    await ws.send_text(msg)
                except Exception:
                    dead.append(ws)
            for ws in dead:
                await self.disconnect(discussion_id, ws)

manager = _WSManager()

@app.websocket("/ws/discussions/{discussion_id}")
async def ws_discussion(websocket, discussion_id):
    await manager.connect(discussion_id, websocket)
    try:
        await websocket.send_text(json.dumps(
            {"type": "hello", "discussion_id": discussion_id}))
        while True:
            await websocket.receive_text() # keep alive, ignore content
    except WebSocketDisconnect:
        pass
    finally:
        await manager.disconnect(discussion_id, websocket)
```

Design notes:

- **Per-thread subscriber set** → broadcast targets only the clients watching that exact discussion. Two threads = two isolated rooms.
- **Async lock** protects the subscriber map during concurrent connect/disconnect.
- **Dead-socket cleanup** → if `send_text` raises, the WebSocket is removed from the room automatically.
- **Hello message** on connect → the frontend uses it to flip the badge from OFFLINE to LIVE.

4.3 Broadcasting on new comment

```
@app.post("/discussions/{discussion_id}/comments", response_model=CommentOut)
async def add_comment(discussion_id: int, req: NewCommentRequest):
    with closing(_db_connect()) as conn:
        with conn.cursor() as cur:
            cur.execute(
                "INSERT INTO discussion_comments (discussion_id, user_id, "
                "author_display, message, is_system) "
                "VALUES (%s, %s, %s, %s, FALSE) "
                "RETURNING comment_id, discussion_id, author_display, "
                "message, is_system, created_at",
                (discussion_id, req.user_id, req.author_display, req.message),
            )
            row = cur.fetchone()
            cur.execute("UPDATE discussions SET updated_at = NOW() "
                        "WHERE discussion_id = %s", (discussion_id,))
            conn.commit()

        comment = CommentOut(...row...)
        await manager.broadcast(
            discussion_id,
            {"type": "comment", "data": comment.model_dump()},
        )
    return comment
```

The DB write happens first, the broadcast after. This way the message is durable even if the WebSocket fan-out fails partway. Late joiners get the data on the next GET; live clients see it instantly via WS.

4.4 Frontend subscription client

```
export const subscribeToDiscussion = (discussionId, onEvent) => {
    const wsUrl = `${getWsBaseUrl()}/ws/discussions/${discussionId}`;
    const state = { closed: false, ws: null, retryMs: 1000, pingTimer: null };

    const open = () => {
        if (state.closed) return;
        const ws = new WebSocket(wsUrl);
        state.ws = ws;

        ws.onopen = () => {
            state.retryMs = 1000;
            state.pingTimer = setInterval(() => {
                if (ws.readyState === WebSocket.OPEN) ws.send('ping');
            }, 25000);
        };

        ws.onmessage = (e) => {
            try { onEvent?.(JSON.parse(e.data)); } catch {}
        };

        ws.onclose = () => {
            if (state.pingTimer) clearInterval(state.pingTimer);
            if (state.closed) return;
            setTimeout(open, state.retryMs);
            state.retryMs = Math.min(state.retryMs * 2, 10000); // capped backoff
        };
    };

    open();
};
```



```

return () => {
  state.closed = true;
  if (state.pingTimer) clearInterval(state.pingTimer);
  if (state.ws && state.ws.readyState <= 1) state.ws.close();
};
};

```

Robustness features baked in:

- **Exponential backoff** (1s → 2s → 4s → 8s → 10s cap) on unexpected disconnect.
- **25s heartbeat ping** → some proxies kill idle WebSockets after ~30s. A periodic send('ping') keeps them alive.
- **Closed flag** → the cleanup function from useEffect sets closed=true so the reconnect loop stops when the component unmounts.

4.5 React integration

```

useEffect(() => {
  if (!incidentId) return;
  setWsConnected(false);

  const unsubscribe = subscribeToDiscussion(incidentId, (event) => {
    if (event?.type === 'hello') {
      setWsConnected(true);
      return;
    }
    if (event?.type === 'comment' && event.data) {
      setSelectedDetail((prev) => {
        if (!prev || prev.discussion_id !== event.data.discussion_id) return prev;
        if (prev.comments.some((c) => c.comment_id === event.data.comment_id)) return prev;
        return { ...prev, comments: [...prev.comments, event.data] };
      });
    }
    if (event?.type === 'status' && event.data) {
      setSelectedDetail((prev) => prev
        ? { ...prev, status: event.data.status }
        : prev);
      reloadList();
    }
  });

  return () => { unsubscribe(); setWsConnected(false); };
}, [incidentId, reloadList]);

```

Idempotent appends → the `some(c => c.comment_id === ...)` check skips duplicates in case both the POST response and the WS broadcast deliver the same comment. Without this, you would see the comment twice in your own client.

5. Computed metrics endpoints

Replaced two frontend mocks (fetchObservabilityMetrics, fetchResolutionData) with real database queries in Python.

5.1 GET /metrics/observability

```
@app.get("/metrics/observability")
def metrics_observability():
    with closing(_db_connect()) as conn:
        with conn.cursor() as cur:
            cur.execute("SELECT MIN(created_at) FROM events")
            first_event = cur.fetchone()[0]

            cur.execute(
                "SELECT COUNT(*) FILTER ("
                "  WHERE LOWER(severity) IN ('critical', 'incident')) AS errors, "
                "  COUNT(*) AS total "
                "FROM events "
                "WHERE created_at >= NOW() - INTERVAL '24 hours'"
            )
            errors, total = cur.fetchone()

    # uptime formatting: Xd Yh / Yh Zm / Zm
    # error rate: ratio of critical+incident in last 24h
    return [
        ObservabilityMetric(id=1, label="UPTIME", value=uptime_str, sublabel=...),
        ObservabilityMetric(id=2, label="ERROR RATE", value=err_str, sublabel=...),
    ]
```

UPTIME — uses MIN(created_at) on events. **ERROR RATE** — FILTER (WHERE ...) is a PostgreSQL aggregate construct that counts only rows matching a condition, in a single pass. Cleaner than COUNT(CASE WHEN ...).

5.2 GET /metrics/resolution

```
@app.get("/metrics/resolution")
def metrics_resolution():
    with closing(_db_connect()) as conn:
        with conn.cursor() as cur:
            cur.execute(
                "SELECT TO_CHAR(date_trunc('day', resolved_at), 'Dy') AS day, "
                "  date_trunc('day', resolved_at) AS d, "
                "  AVG(EXTRACT(EPOCH FROM (resolved_at - created_at)) / 60.0) "
                "FROM events "
                "WHERE resolved_at IS NOT NULL "
                "  AND resolved_at >= NOW() - INTERVAL '7 days' "
                "GROUP BY d, day "
                "ORDER BY d ASC"
            )
            rows = cur.fetchall()
    return [ResolutionPoint(day=r[0].strip(), time=float(r[2] or 0)) for r in rows]
```

EXTRACT(EPOCH FROM interval) → converts a duration to seconds; we divide by 60 to get minutes.
date_trunc('day', ...) → groups multiple events resolved the same day together, regardless of time.

5.3 Frontend wiring

```
// src/services/api.js
import { getChatApiBaseUrl } from './chatApi';

export const fetchObservabilityMetrics = async () => {
    try {
        const res = await fetch(`${getChatApiBaseUrl()}/metrics/observability`);
        if (!res.ok) throw new Error(`HTTP ${res.status}`);
        return await res.json();
    } catch (err) {
        console.error('fetchObservabilityMetrics failed', err);
    }
}
```

```
    return [
      { id: 1, label: 'UPTIME',      value: 'OFFLINE', sublabel: 'Backend down' },
      { id: 2, label: 'ERROR RATE', value: 'N/A',      sublabel: 'Backend down' },
    ];
  }
};
```

6. Bug fixes and stabilization

6.1 ChartWidget hover crash

Symptom: Hovering over the Thermal Metrics chart on Dashboard caused the whole page to go black with `ReferenceError: seriesName is not defined`.

Cause: `CustomTooltip` was defined outside the `ChartWidget` closure but tried to reference variables (`seriesName`, `unit`) that only existed inside the parent component.

```
// Before (broken):
const CustomTooltip = ({ active, payload, label }) => {
  // ...
  {payload.map((entry) => (
    <Typography>{seriesName}: {entry.value}{unit}</Typography>
    ^^^^^^^^^^^^ undefined here!
  ))}
};

// After (fixed):
const CustomTooltip = ({ active, payload, label, unit = '' }) => {
  {payload.map((entry) => (
    <Typography>{entry.name || 'VALUE': {entry.value}{unit}</Typography>
  ))}
};
// Recharts populates entry.name automatically from <Line name='...' />
```

6.2 Dashboard duplicate declaration

Symptom: Vite parse error: Expected `}` but found `EOF`.

Cause: An import was inserted in the middle of the function body and the component declaration was duplicated:

```
// Broken file:
const Dashboard = () => {                                // first declaration
  const [params, patchParams] = useUrlState();
  import { COLORS } from '../theme/colors';              // import inside function!

  const Dashboard = ({ setActivePage }) => {              // second declaration
    // ...
  }
}
```

Fix: Moved the import to the top of the file, kept a single declaration accepting `{ setActivePage }`.

6.3 ErrorBoundary as safety net

Added a class component that catches any render error in the React tree, logs the stack to console and renders a friendly recovery card (*Try Again, Reload, Logout + Reset*) instead of leaving the user with a blank page.

```
class ErrorBoundary extends React.Component {
  state = { error: null, info: null };

  static getDerivedStateFromError(error) { return { error }; }

  componentDidCatch(error, info) {
    console.error(`[ErrorBoundary / ${this.props.scope}]`, error, info);
    this.setState({ info });
  }

  reset = () => this.setState({ error: null, info: null });

  render() {
    if (!this.state.error) return this.props.children;
    return <FallbackCard error={this.state.error} onReset={this.reset} />;
  }
}
```

```
// Mounted twice:
//   - main.jsx wraps <App /> entirely (root scope)
//   - App.jsx wraps each page (scope = page name)
// The page-level boundary uses key={activePage} so navigating to a
// different page also resets a previous error state.
```

6.4 Polling intervals stabilization

Multiple components polled every 5 seconds. With the backend slow or down, requests piled up and made the browser unresponsive ("site crap■ foarte u■or"). Doubled or tripled all intervals.

Component	Before	After
Dashboard.jsx	5s	30s
Tickets.jsx	5s	30s
SeverityPieChart.jsx	5s	30s
AlarmFrequencyChart.jsx	15s	60s
Layout.jsx notifications	15s	60s

7. UI improvements and humanization

7.1 Dashboard cards are now clickable

```
const CARD_TARGET = {
  'CPU TEMP': 'Observability',
  'OPEN TICKETS': 'Tickets',
  'TOTAL EVENTS': 'Tickets',
  'SENSOR ID': 'Devices',
};
const target = CARD_TARGET[metric.title];
const clickable = !!target && typeof setActivePage === 'function';
<Card
  onClick={clickable ? () => setActivePage(target) : undefined}
  sx={{ cursor: clickable ? 'pointer' : 'default', ... }}
>
```

7.2 Text rewrite — from AI marketing to human

Every page subtitle and most headings sounded like generated SaaS marketing copy. Replaced with short, human phrasing.

Before	After
Incident Control Panel	Overview
Device Management	Devices
Ticket Management	Open Tickets
Metrics & Logs	Observability
Command Center	NOC Wall
Incident Discussions	Discussions
Provision, monitor, and manage your infrastructure assets	Sensors, locations, and physical units in the network.
System telemetry, active KPIs, and the tail of recent events	Events metrics and the latest log entries.
Active system alerts requiring operator intervention and acknowledgment	Open alerts requiring to be acknowledged.
Drag & drop to rearrange active monitoring feeds.	Drag the panels to rearrange.
Conversational AI for telemetry, incidents and database insights	Ask about sensors, incidents, or the database.
Centralized forum for investigating and documenting system status	System status incidents. Add comments, change status.
GLOBAL_TELEMETRY (sidebar label)	System Stats

7.3 Login screen polish

- Background changed to solid gray (#232323) — cleaner than the white default.
- Field labels are always visible above their inputs (InputLabelProps={{ shrink: true }}) instead of waiting for click.
- Enter submits the form (wrapped in <Box component='form' onSubmit={handleSubmit}> + button type='submit').
- URL stays in sync: ?page=Login when logged out, ?page=Dashboard after login.
- Removed dead UI: *Remember me* checkbox, *Forgot password?* link, *SSO/IDP Provider* button and *v0.1.0 MOCK* footer — none of them did anything.

7.4 Profile menu shows real user

Layout.jsx receives the user prop from App.jsx and renders:

- Email + role chip next to the avatar (top right)
- Inside the popover: full name + email + role + green dot
- Red **Logout** action that calls onLogout, which clears `sessionStorage.nexus_user` and triggers the app to re-render the Login screen.

8. Helper CLI scripts

Three small scripts make the system bootstrap and user management painless. All read DB_* values from .env.

8.1 set_password.py

```
# Usage: python set_password.py <email> <new_password>
# Example: python set_password.py admin@infrapulse.com admin123

email = sys.argv[1].lower().strip()
password = sys.argv[2]
pw_hash = bcrypt.hashpw(password.encode('utf-8'), bcrypt.gensalt()).decode('utf-8')

cur.execute(
    "UPDATE users SET password_hash = %s, updated_dtm = NOW() "
    "WHERE email = %s RETURNING user_id",
    (pw_hash, email),
)
```

8.2 create_user.py

```
# Usage: python create_user.py <email> <pass> <first> [last] [role]
# Example: python create_user.py john@x.com pwd123 John Doe user

email      = sys.argv[1].lower().strip()
password   = sys.argv[2]
first_name = sys.argv[3]
last_name  = sys.argv[4] if len(sys.argv) > 4 else ''
role       = sys.argv[5] if len(sys.argv) > 5 else 'user'

pw_hash = bcrypt.hashpw(password.encode(), bcrypt.gensalt()).decode()

cur.execute(
    "SELECT user_id FROM users WHERE email = %s", (email,)
)
if cur.fetchone():
    sys.exit(2) # already exists

cur.execute(
    "INSERT INTO users (first_name, last_name, email, role, "
    "password_hash) VALUES (%s, %s, %s, %s, %s) RETURNING user_id",
    (first_name, last_name, email, role, pw_hash),
)
```

8.3 seed_discussions.py

Idempotent — if the table already has rows it bails out. Inserts three threads + their comments. All linked to creator_user_id = 1 (admin).

```
for d in SEED:
    cur.execute(
        "INSERT INTO discussions (title, status, author_display, "
        "device_label, body, creator_user_id) "
        "VALUES (%s, %s, %s, %s, %s, 1) RETURNING discussion_id",
        (d['title'], d['status'], d['author_display'],
         d['device_label'], d['body']),
    )
    did = cur.fetchone()[0]
    for author, is_system, msg in d['comments']:
        cur.execute(
            "INSERT INTO discussion_comments (discussion_id, "
            "author_display, message, is_system) "
            "VALUES (%s, %s, %s, %s)",
            (did, author, msg, is_system),
        )
    )
```

9. Setup and run instructions

9.1 First-time setup

```
# 1. Install Python dependencies
python -m pip install bcrypt reportlab
# (psycpg2-binary, fastapi, uvicorn, websockets, dotenv
# were already in requirements.txt)

# 2. Make sure .env has DB_PORT=5433 (Docker external port)
# .env example:
#   DB_HOST=localhost
#   DB_PORT=5433
#   DB_USER=postgres
#   DB_PASSWORD=postgres
#   DB_NAME=nexus
#   VITE_CHAT_API_BASE_URL=http://127.0.0.1:8002

# 3. Start Postgres
docker compose up -d postgres

# 4. Apply migration 003 (one-time, since it was added after first start)
docker exec -i nexus-postgres psql -U postgres -d nexus < migrations/003_discussions.sql

# 5. Seed initial discussion threads
python seed_discussions.py

# 6. Set a real password for the seeded admin
python set_password.py admin@infrapulse.com admin123
```

9.2 Daily run

```
# Terminal 1 - Python API (login, discussions, metrics, chatbot, WS)
python -m uvicorn chat_api:app --reload --port 8002

# Terminal 2 - frontend
npm run dev
```

Open <http://localhost:5173> → you land on the login screen automatically because `App.jsx` detects no `nexus_user` in `sessionStorage`.

10. Demo / presentation script

Suggested flow for showing the project. Each step takes ~10-15 seconds.

Step 1 — Auth gating

Show that going to `localhost:5173` immediately redirects to `?page=Login`. Try typing a wrong password → red *Invalid credentials*. Type the right one → Dashboard loads, email appears top-right.

Step 2 — Sign up a new user

Click *Don't have account? Sign up* → fill in first/last name, email, password → click *Create account*. The form returns to login mode with a green *Account created. Please sign in.* message. Confirm in the DB:

```
docker exec -it nexus-postgres psql -U postgres -d nexus \
-c "SELECT user_id, email, role, created_at FROM users ORDER BY user_id;"
```

Step 3 — Real-time discussions (the wow moment)

- Open **two browser windows** side by side (one normal, one private) → log in as two different users.
- Both navigate to `?page=Discussions&incident;=1`.
- Notice the green **LIVE** badge in the top right (WebSocket connected).
- Type a comment in window A → it appears **instantly** in window B.
- Click *Close Incident* in window B → status chip flips from **OPEN** to **RESOLVED** in both windows, plus a system comment is auto-inserted.

Closing the demo: open the DB and show the persisted comments — nothing was ephemeral.

```
docker exec -it nexus-postgres psql -U postgres -d nexus -c \
"SELECT comment_id, user_id, author_display, LEFT(message, 50) AS msg \
FROM discussion_comments \
ORDER BY created_at DESC LIMIT 10;"
```

Step 4 — Real metrics, not hardcoded

Navigate to *Observability*. The UPTIME and ERROR RATE values come directly from events. Show the queries in `chat_api.py` (sections 5.1 / 5.2 of this doc).

Step 5 — Dashboard cards are interactive

Click each Dashboard card → navigates to the relevant page. Quick to demonstrate.

Step 6 — Logout, then session persistence

Log out (red button in profile menu) → back to login. Login again, navigate, then press F5: you stay logged in (sessionStorage). Close the tab and reopen: logged out (sessionStorage is per-tab).

11. Security notes and known limitations

What is in place:

- Passwords stored as bcrypt hashes (work factor 12). Plaintext never touches the DB or logs.
- Parameterized SQL with %s placeholders → SQL injection safe.
- Generic *Invalid credentials* error → cannot enumerate accounts by error message.
- CORS allow-list restricted to specific origins, not *.
- WebSocket disconnects are cleaned up on both client and server.
- ErrorBoundary catches render errors instead of exposing a blank page.

Known limitations (intentional, for the prototype scope):

- **No JWT** — the session is just a flag in sessionStorage. A real deployment would want a signed token sent in the Authorization header on every request, validated server-side.
- **No rate limiting** — brute-force protection isn't implemented. Add slowapi or a failed_login_attempts column for production.
- **No 2FA / email verification** — signup trusts whatever email you type.
- **Forgot password is removed but not replaced** — needs an email service + a password_reset_tokens table.
- **WebSocket has no auth** — anyone who knows a discussion_id can subscribe. For production, require a token on connect.
- **Dev runs on HTTP** — production must use HTTPS/WSS, otherwise passwords and tokens travel in clear.

12. File-by-file changelog

Sorted by area. Skip rows you don't need to mention in the presentation.

Backend

File	Type	Summary
chat_api.py	Extended	+ /login, /signup endpoints with bcrypt + DB. + 5 discussions endpoints. + WebSocket
set_password.py	New	CLI: bcrypt-hash a new password and UPDATE the user row.
create_user.py	New	CLI: INSERT a new user with chosen role.
seed_discussions.py	New	CLI: idempotent seeder for the initial three threads + their comments.
migrations/003_discussions.py	Applied	Pre-existing in the repo but not in the DB; applied via psql.

Frontend — services

File	Type	Summary
src/services/authApi.js	New	fetch helpers loginUser / signupUser; unified JSON error extraction.
src/services/discussionsApi.js	New	fetchDiscussions / fetchDiscussionDetail / createDiscussion / postComment / o
src/services/api.js	Edited	Added getCurrentUser / getCurrentUserId helpers. Replaced fetchObservabili

Frontend — pages and components

File	Type	Summary
src/App.jsx	Edited	User state from sessionStorage + isAuthenticated gate. handleLogin / handleLog
src/main.jsx	Edited	Wraps <App /> in <ErrorBoundary scope='root' />.
src/pages/Login.jsx	Edited	Real fetch-based handleSubmit, mode toggle (login/signup), success/error
src/pages/Discussions.jsx	Rewritten	Fetches from backend, WebSocket subscription per opened thread, LIVE A
src/pages/Dashboard.jsx	Edited	Combined duplicate declaration. CARD_TARGET map makes each KPI ca
src/components/Layout.jsx	Edited	Accepts user + onLogout props. Email + role next to avatar. Logout (red) r
src/components/ErrorBoundary.jsx	New	Class component, getDerivedStateFromError + componentDidCatch, fallba
src/components/ChartWidget.jsx	Fixed	CustomTooltip no longer references out-of-scope variables; uses entry.na
src/components/AlarmFrequencyChart.jsx	Edited	Fixed + other polling intervals bumped (5s -> 30s, 15s -> 60s) to keep the browser resp

Config

File	Change
.env	DB_PORT 5432 -> 5433 (Docker external mapping).
docker-compose.yml	Untouched.
vite.config.js	Untouched.

requirements.txt	No source change, but bcrypt + reportlab were pip-installed.
------------------	--

Document generated programmatically from the actual implementation. Update generate_session_doc.py if you change the source files; then rerun to refresh the PDF.